

EVIDENCIA DE CONOCIMIENTO

Bases teóricas de estructuras de almacenamiento en memoria

GA3-220501093-AA3-EV01

CAMPO	INFORMACIÓN
Programa	Análisis y Desarrollo de Software
Institución	Servicio Nacional de Aprendizaje SENA
Fase	Planeación
Naturaleza	Documento académico público

VERSIÓN PÚBLICA SIN DATOS PERSONALES

Lenguajes · JavaScript · Tipos primitivos · Operadores

00 · PUNTO DE PARTIDA

Propósito y ruta de lectura

Los cinco indicadores se atienden de forma explícita y verificable.

PROPÓSITO

Consolidar las bases necesarias para leer y escribir JavaScript con criterio: comprender cómo se ejecuta un lenguaje, qué caracteriza a ECMAScript, cómo se representan los valores simples y cómo los operadores transforman esos valores.

Objetivos específicos

- Comparar compilación e interpretación sin presentar categorías absolutas.
- Distinguir el lenguaje JavaScript del motor y de las API ofrecidas por cada entorno.
- Reconocer los siete primitivos vigentes y seleccionar su uso de acuerdo con el dato.
- Aplicar operadores atendiendo a precedencia, coerción y evaluación por cortocircuito.

Ruta del documento

CAPÍTULO	PREGUNTA QUE RESUELVE	APOYO VISUAL
1	¿Cómo llega el código fuente a ejecutarse?	Dos canales de ejecución.
2	¿Qué hace distintivo a JavaScript?	Capas lenguaje–motor–entorno.
3	¿Qué valores primitivos existen y para qué sirven?	Mapa de siete tipos.
4	¿Cómo se forman y evalúan expresiones?	Escalera de precedencia.

Método: síntesis comparativa de la guía SENA [1], la especificación ECMA-262 [2], MDN [3–6] y la documentación del motor V8 [7]. Las figuras son elaboración propia y declaran su base conceptual.

01 · ESTRATEGIAS DE EJECUCIÓN

Lenguajes compilados e interpretados

La diferencia principal está en cuándo y cómo se traduce el programa.

Compilar significa transformar un programa a otro formato antes o durante su ejecución. En un flujo AOT (ahead of time), la traducción suele producir un archivo ejecutable antes de iniciar el programa. Interpretar significa que un programa intermediario analiza y ejecuta instrucciones durante la marcha. Son estrategias de implementación: un lenguaje no queda obligado para siempre a una sola de ellas [5].

ASPECTO	COMPILACIÓN AOT	INTERPRETACIÓN
Momento	Traducción previa a la ejecución.	Análisis y ejecución durante el uso.
Producto	Puede generar binario o código objeto.	Normalmente conserva el fuente y usa un intérprete.
Errores	Muchos se detectan al compilar.	Algunos aparecen al alcanzar la instrucción.
Inicio	Requiere una fase previa de construcción.	Puede iniciar sin crear un ejecutable independiente.
Rendimiento	El ejecutable ya está traducido.	La traducción añade trabajo en tiempo de ejecución.
Portabilidad	El binario depende del destino.	El fuente viaja si existe un entorno compatible.

CONCLUSIÓN COMPARATIVA

Compilado no significa automáticamente mejor, ni interpretado significa necesariamente lento. Importan el motor, la optimización, la carga de trabajo y el momento en que se realiza la traducción.

01 · ESTRATEGIAS DE EJECUCIÓN

El caso real de JavaScript

Los motores modernos combinan intérprete y compilador JIT.

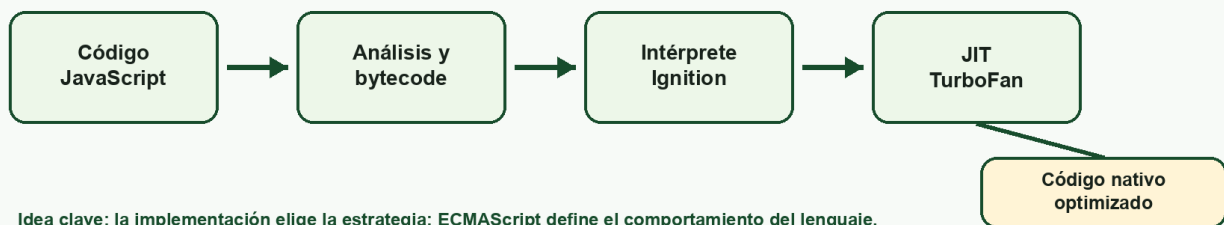
FIGURA 1 · ESTRATEGIAS DE EJECUCIÓN

Compilar e interpretar describen rutas de trabajo; un motor puede combinarlas.

COMPILACIÓN AOT



JAVASCRIPT MODERNO



Idea clave: la implementación elige la estrategia; ECMAScript define el comportamiento del lenguaje.

Figura 1. Estrategias de ejecución. Elaboración propia con base en MDN [5] y V8 [7].

ECMAScript especifica la sintaxis y los resultados observables, pero no obliga a una arquitectura interna. En V8, el código pasa por análisis y bytecode, se ejecuta con Ignition y las partes usadas con frecuencia pueden optimizarse con TurboFan. Por eso es más preciso afirmar que JavaScript suele ejecutarse mediante una combinación de interpretación y compilación JIT [7].

AOT FRENTE A JIT

AOT traduce antes de iniciar; JIT compila durante la ejecución usando información real del programa. Una misma aplicación puede recorrer bytecode y código nativo optimizado.

02 · LENGUAJE

Características principales de JavaScript

JavaScript está estandarizado mediante la especificación ECMAScript.

CARACTERÍSTICA	SIGNIFICADO	IMPLICACIÓN
Estandarizado	ECMA-262 define la semántica.	Los motores comparten una base común.
Dinámico	La variable no queda unida a un tipo.	El tipo pertenece al valor y puede cambiar.
Coerción	Admite conversiones implícitas.	Conviene controlar comparaciones y entradas.
Multiparadigma	Combina estilos imperativo, funcional y orientado a objetos.	Permite escoger una solución adecuada al problema.
Prototipos	Los objetos delegan mediante cadenas de prototipos.	La herencia no depende solo de clases.
Primera clase	Las funciones son valores invocables.	Pueden pasarse, retornarse y almacenarse.
Memoria automática	El motor recupera objetos no alcanzables.	No se libera memoria manualmente.
Portable	Funciona en distintos anfitriones.	La API disponible depende del entorno.

PRECISIÓN TERMINOLÓGICA

JavaScript es el lenguaje; ECMAScript es su especificación; V8, SpiderMonkey o JavaScriptCore son motores; navegador y Node.js son entornos que añaden API.

02 · LENGUAJE

Tres capas que conviene separar

Esta distinción evita atribuir al lenguaje funciones que pertenecen al entorno.

FIGURA 2 · LENGUAJE, MOTOR Y ENTORNO

JavaScript no es sinónimo del navegador: cada capa tiene una responsabilidad.

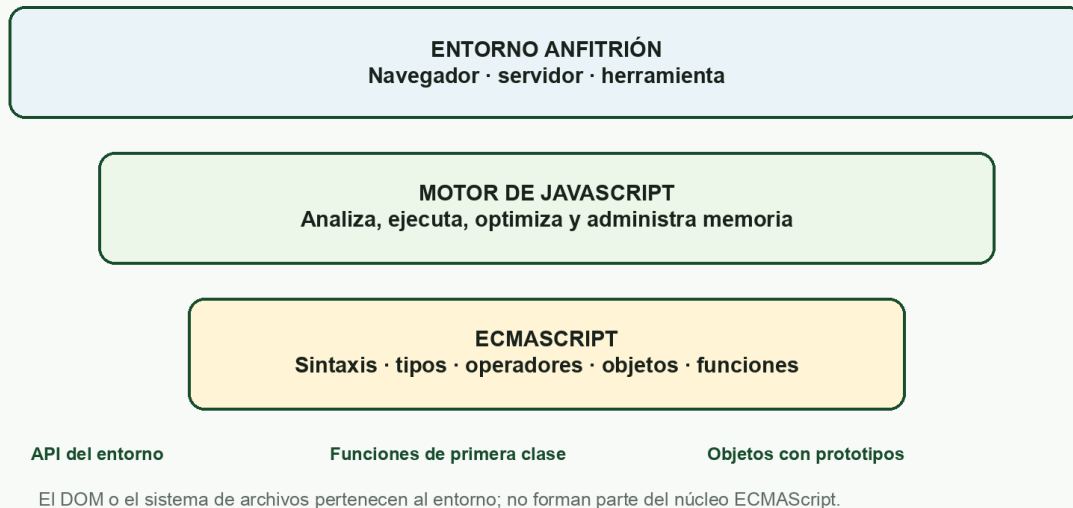


Figura 2. Capas de una ejecución JavaScript. Elaboración propia con base en ECMA-262 [2] y MDN [3].

El núcleo ECMAScript aporta valores, operadores, control, objetos, funciones, clases y módulos. El motor materializa esas reglas y se ocupa de ejecutar y optimizar. El anfitrión agrega capacidades: un navegador ofrece el DOM y eventos; un entorno de servidor puede ofrecer sistema de archivos o red. Esas API no forman parte del lenguaje [2, 3].

```
const duplicar = (valor) => valor * 2;
const operaciones = [duplicar];
console.log(operaciones[0](21)); // 42
```

El ejemplo muestra una función como valor: se guarda dentro de un arreglo y después se invoca. Esa propiedad sustenta callbacks, composición y gran parte de la programación asíncrona.

03 · VALORES

Tipos de datos primitivos

ECMA-262 define siete tipos primitivos y *Object* como categoría separada.

TIPO	EJEMPLO	USO HABITUAL	typeof
String	"Bogotá"	Texto, etiquetas y entradas.	"string"
Number	42; 3.14; NaN	Cálculo general con enteros o decimales.	"number"
BigInt	9007199254740993n	Enteros fuera del rango seguro de Number.	"bigint"
Boolean	true; false	Decisiones y estados lógicos.	"boolean"
Undefined	let dato;	Ausencia de asignación o retorno.	"undefined"
Symbol	Symbol("clave")	Identificadores únicos de propiedades.	"symbol"
Null	null	Ausencia intencional de un valor de objeto.	"object"*

Un primitivo es inmutable: la operación crea otro valor en vez de modificar el existente. La variable sí puede reasignarse. Arreglos, funciones, fechas y objetos pertenecen a *Object*, por lo que no deben sumarse como un octavo primitivo [2, 4].

BIGINT Y NUMBER

No deben mezclarse directamente en aritmética. *BigInt* conserva enteros arbitrariamente grandes, pero no representa fracciones. *Number* usa punto flotante y tiene límites de precisión.

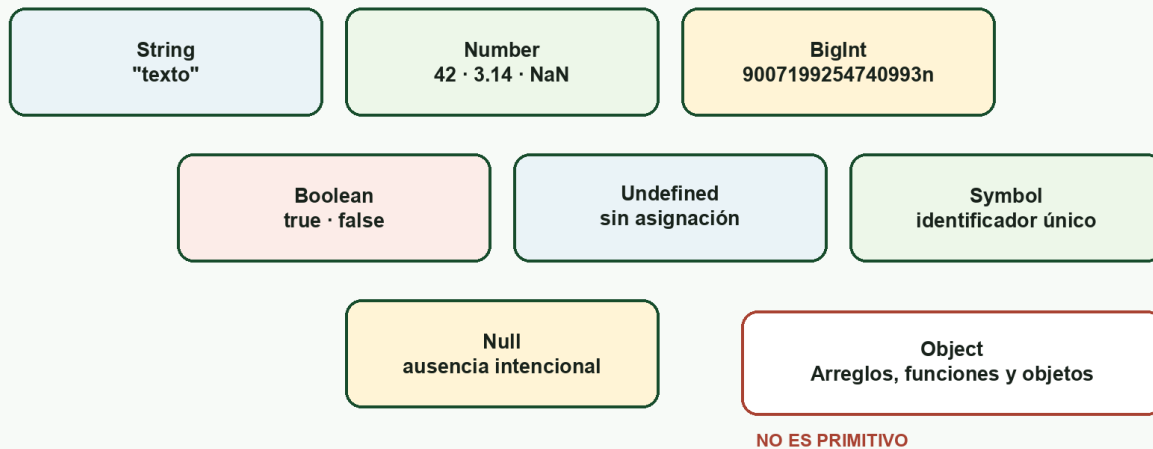
03 · VALORES

Uso, detección y dos excepciones

El tipo correcto comunica intención y evita conversiones inesperadas.

FIGURA 3 · LOS SIETE PRIMITIVOS

Son valores inmutables; Object constituye la categoría no primitiva.



Atención: `typeof null` produce "object" por compatibilidad histórica; para detectarlo se usa valor `=== null`.

Figura 3. Mapa de tipos primitivos. Elaboración propia con base en ECMA-262 [2] y MDN [4].

```
const titulo = "Informe";           // String
const paginas = 12;                 // Number
const publicado = true;             // Boolean
let observacion;                   // Undefined
const seleccion = null;             // Null
const clave = Symbol("registro");   // Symbol
```

- `typeof null` devuelve "object" por compatibilidad histórica; la comprobación segura es `valor === null`.
- NaN pertenece al tipo Number, pero representa un resultado numérico inválido; se detecta con `Number.isNaN(valor)`.
- Una propiedad inexistente suele producir undefined; null suele expresar una ausencia decidida por el programa.

04 · EXPRESIONES

Operadores en JavaScript

Un operador recibe operandos y produce un valor o un efecto.

FAMILIA	OPERADORES	EJEMPLO Y RESULTADO
Aritméticos	+ - * / % **	2 ** 3 → 8
Asignación	= += -= *= ??=	total += 5
Comparación	< > <= >=	edad >= 18 → booleano
Igualdad	=== !== == !=	5 === "5" → false
Lógicos	&& !	activo && autorizado
Fusión nula	??	nombre ?? "Sin dato"
Condicional	? :	mayor ? "Sí" : "No"
Unarios	typeof delete void ++	typeof 42 → number
Relacionales	in instanceof	"id" in objeto
Bit a bit	& ^ ~ << >> >>>	5 & 1 → 1

IGUALDAD ESTRICTA

`===` y `!==` comparan sin convertir tipos y suelen expresar mejor la intención. La igualdad flexible puede ser válida, pero exige conocer con precisión sus reglas de coerción [6].

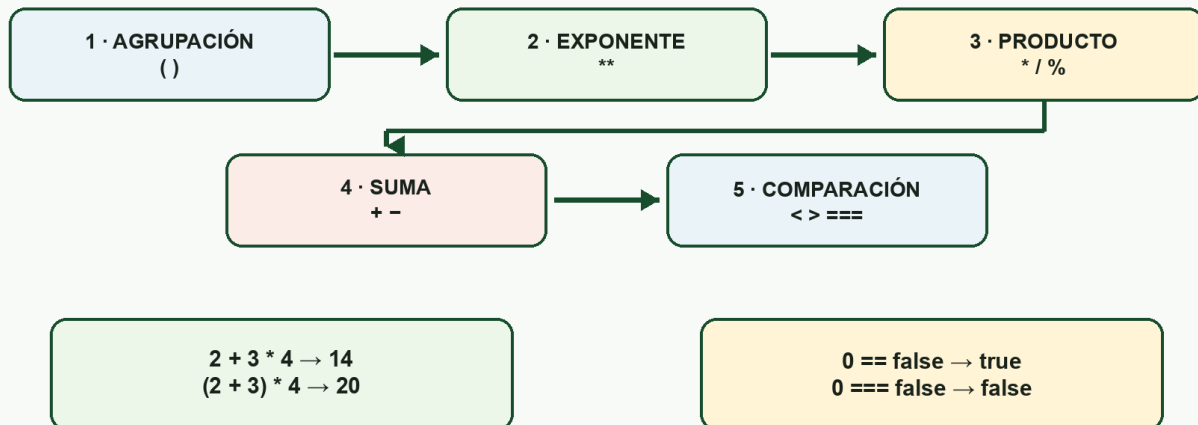
04 · EXPRESIONES

Precedencia, coerción y cortocircuito

Los paréntesis y la igualdad estricta vuelven el resultado más predecible.

FIGURA 4 · OPERADORES Y ORDEN DE EVALUACIÓN

La precedencia organiza la expresión; los paréntesis hacen explícita la intención.



Recomendación: igualdad estricta, paréntesis cuando aclaren la regla y cortocircuito usado conscientemente.

Figura 4. Orden de evaluación. Elaboración propia con base en MDN [6].

```

const subtotal = 85000;
const descuento = subtotal > 50000 ? subtotal * 0.20 : 0;
const total = subtotal - descuento; // 68000

const alias = null;
const etiqueta = alias ?? "Sin registrar";
  
```

- La multiplicación se evalúa antes que la suma; los paréntesis pueden cambiar o aclarar ese orden.
- `"5" + 2` produce `"52"`; `Number("5") + 2` produce `7`. La conversión explícita comunica la intención.
- `&&`, `||` y `??` usan cortocircuito: el segundo operando se evalúa solo cuando la regla lo necesita.

05 · CIERRE

Conclusiones y matriz de cumplimiento

El conocimiento conceptual se traduce en decisiones de código más seguras.

- Compilación e interpretación son estrategias combinables; JavaScript moderno usa ejecución adaptativa.
- La especificación, el motor y el entorno son capas distintas y complementarias.
- Los siete primitivos representan valores simples e inmutables; Object agrupa las estructuras compuestas.
- Los operadores requieren considerar tipo, precedencia, coerción y cortocircuito.

Cobertura de la lista de chequeo

N.º	INDICADOR	EVIDENCIA	ESTADO
1	Compilados e interpretados	Páginas 3–4, tabla y figura 1.	Cumple
2	Características de JavaScript	Páginas 5–6, tabla, figura 2 y código.	Cumple
3	Tipos primitivos y uso	Páginas 7–8, tabla, figura 3 y ejemplos.	Cumple
4	Operadores	Páginas 9–10, tabla, figura 4 y ejemplos.	Cumple
5	Imágenes y fuentes	Cuatro figuras propias y referencias numeradas.	Cumple

RESULTADO

Los cinco indicadores tienen evidencia directa. El documento evita extender el alcance hacia los ejercicios de programación que corresponden a la evidencia posterior.

06 · FUENTES

Referencias

Consultadas el 16 de agosto de 2026. Enlaces directos a la fuente utilizada.

- [1] Servicio Nacional de Aprendizaje SENA. (s. f.). Guía de aprendizaje: fase de planeación, actividad GA3-220501093-AA3, pp. 8–9.
https://archivos.territorio.la/archivos/clases/Guianaprendizajen3____58631be32843215____.pdf
- [2] Ecma International. (2026). ECMAScript Language Specification (ECMA-262). <https://tc39.es/ecma262/>
- [3] MDN Web Docs. (2025). JavaScript language overview.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview
- [4] MDN Web Docs. (2025). JavaScript data types and data structures.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures
- [5] MDN Web Docs. (2025). Compile. <https://developer.mozilla.org/en-US/docs/Glossary/Compile>
- [6] MDN Web Docs. (2025). Expressions and operators.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators
- [7] V8 Project. (2017). Launching Ignition and TurboFan. <https://v8.dev/blog/launching-ignition-and-turbofan>

Procedencia de las ilustraciones

Las figuras 1–4 son elaboración propia generada específicamente para esta evidencia. No contienen imágenes, logotipos ni capturas de terceros. Sus pies señalan las fuentes conceptuales que sustentan cada representación.

CONTROL DE VERSIÓN

La edición pública y la edición completa local comparten exactamente este desarrollo académico. Únicamente cambian la portada y los metadatos del documento.